InfraQ

A Scheduling Gateway for Shared Local LLM Inference

github.com/bowenwan6/InfraQ

InfraQ brings vLLM-style scheduling to Mac hardware, without CUDA or NVIDIA. Four capabilities make this work: a RabbitMQ queue that decouples client acceptance from inference so requests are never dropped under burst load; a K-slot parallel scheduler exploiting server-side concurrency; an exact-match Redis cache returning repeated prompts in ∼11 ms; and a Redis config channel for zero-restart strategy switching. On repeated-prompt workloads, cache-aware dispatching achieved ***5.6× higher throughput*** *and* ***76% lower average latency****; on unique-prompt workloads (`OLLAMA_NUM_PARALLEL=4`, 3 repeats), parallel scheduling achieved* ***60% higher throughput*** *over sequential (0.368 vs. 0.230 req/s) and* ***38% lower P95*** *(252 s vs. 409 s), with continuous dispatching edging static batching by eliminating batch-boundary stalls.*

Problem and Idea

Small engineering teams increasingly run LLMs on a shared Mac server instead of cloud APIs. Without a message queue, concurrent requests pile up and latency spikes unpredictably; even with one, a serial consumer still leaves server-side concurrency idle. Two more problems compound this: repeated prompts are re-inferred from scratch every time, and changing the scheduling strategy requires a container restart.

vLLM [1] and SGLang [2] solve these at the GPU kernel level—continuous batching and KV-cache prefix reuse—but both require NVIDIA hardware that most small teams cannot justify.

InfraQ re-implements both techniques at the infrastructure layer, making them available on any local LLM server (Ollama [3], LM Studio, llama.cpp), including Mac M1—no NVIDIA required. The four capabilities in the abstract are the result.

Scheduling Strategies

InfraQ implements four strategies, each adding exactly one new mechanism so its effect can be isolated in the experiments of §4. Table 1 summarises them.

Table 1: Scheduling strategies in InfraQ.

Strategy	Mechanism	Inspiration	Trade-off addressed
Sequential	One request at a time; no parallelism	Baseline	None — serial baseline
Static batching	Collect N requests, dispatch all, wait for batch to finish	Classic GPU batching	Slowest request stalls free slots
Continuous dispatching	Maintain K slots; refill immediately when any slot completes	vLLM continuous batching	Eliminates batch-boundary stalls; keeps slots filled
Cache-aware dispatching	Redis SHA-256 lookup per slot; hits return ∼11 ms, skipping Ollama	SGLang prefix caching	Near-zero latency on cache hits; frees slots for misses

Architecture

The Spring Boot gateway returns HTTP 202 immediately, then writes to PostgreSQL, caches job status in Redis, and enqueues to RabbitMQ—none of these block on inference, so the gateway remains responsive under burst load. Spring Boot was chosen for its production-grade embedded HTTP server and dependency injection; PostgreSQL for its durable, queryable record store that persists across container restarts. The Python asyncio worker consumes from RabbitMQ and applies the active scheduling strategy. Ollama runs natively on the host, reached via `host.docker.internal`, preserving Mac Metal GPU acceleration without container passthrough.

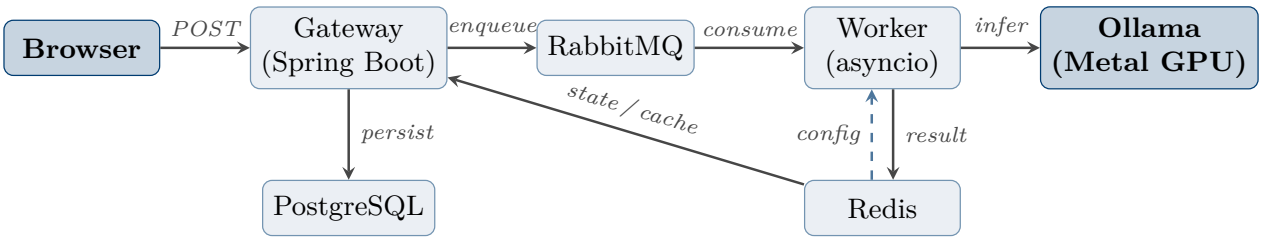

Figure 1: InfraQ architecture. Dashed arrow: Redis config channel propagating live strategy changes to the worker without restart.

Redis serves four roles: browser-polling fast-path (<1 ms), prompt-result cache, atomic metric counters, and the live configuration channel (dashed in Figure 1).

Experimental Results

Experiments ran on MacBook Air M1 (8 GB) using Qwen2.5:1.5b, 100 requests per run; full setup in Appendix A. Raw data in `summary.csv` and `manifest.json`.

Scheduling without caching (unique prompts)

Table 2: Exp. A: unique prompts, OLLAMA_NUM_PARALLEL=4, 3-run averages.

Strategy	Slots	Avg latency	P95	Throughput
Sequential	1	174 s	409 s	0.230 req/s
Static batching	8	110 s	263 s	0.364 req/s
Continuous	8	104 s	**252 s**	**0.368 req/s**
Cached	8	108 s	254 s	0.370 req/s

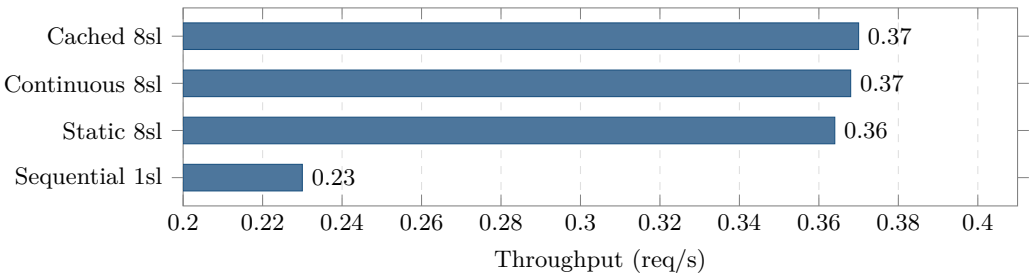

Figure 2: Throughput, Exp. A (OLLAMA_NUM_PARALLEL=4). Parallel strategies are 60% faster than sequential; continuous dispatching edges static batching by 1% by eliminating batch-boundary stalls.

With all prompts unique the cache cannot fire, isolating the scheduler. With `OLLAMA_NUM_PARALLEL=4` enabling true concurrent inference, parallel strategies reach 0.364–0.370 req/s versus 0.230 req/s for sequential—a 60% throughput gain. Continuous dispatching edges static batching by 1% throughput and 4% P95 (252 s vs. 263 s): it refills a slot the moment any request completes, whereas static batching waits for the whole batch before freeing any slot.

Cache-aware dispatching on repeated prompts

Table 3: Exp. B: repeated prompts (hot set), 5-run averages.

Strategy	Slots	Avg latency	P95	Throughput	Hit rate
Sequential	1	311 s	584 s	0.165 req/s	0%
Static batching	4	298 s	557 s	0.173 req/s	0%
Continuous	4	299 s	565 s	0.170 req/s	0%
Cached	**4**	**73 s**	**96 s**	**0.961 req/s**	**71%**

With 71% hitting the Redis cache, cache-aware dispatching delivers a 5.6× throughput increase (0.961 vs. 0.170 req/s) and 76% lower latency (73 s vs. 299 s). With only 29 in 100 reaching Ollama, P95 tail latency collapses from ∼565 s to 96 s (83%). A slot-count ablation (Appendix B) shows 4 slots outperforms 8 under caching: more concurrency is not always better.

Implementation

The worker implements K-slot concurrency as K independent asyncio coroutines, each pulling jobs from RabbitMQ—asyncio gives I/O-bound parallelism without thread-pool overhead. On a strategy change, in-flight slots complete before switching; no request is dropped.

The benchmark framework records `summary.csv`, `manifest.json`, and per-run JSON artifacts, making every result in §4 reproducible from the submitted data. Deployment is `docker compose up`: all five services start in health-checked dependency order with Ollama on the host as the only external requirement. The dashboard covers the full usage loop: chat, live metrics, benchmark lab, and runtime strategy switching.

The caching layer is prototype-grade (exact-match text, not KV tensors); a production system would use embedding-based semantic matching.

AI Usage

All ideas, architecture, and experimental design are my own. Claude Code scaffolded the Spring Boot gateway and generated the HTML/JS dashboard; ChatGPT helped debug the Python asyncio worker and scheduling loop. The benchmark design, methodology, and conclusions are entirely mine.

Appendix A: Experimental Setup

All experiments ran on a MacBook Air M1 (8 GB) using Qwen2.5:1.5b via Ollama, 100 requests per run. Experiment A used `OLLAMA_NUM_PARALLEL=4` (enabling true concurrent inference) with 3 repeats; Experiment B used default Ollama settings (serial inference) with 5 repeats. Unique-prompt runs drew 100 distinct prompts (40 topics $\times$ 4 templates); repeated-prompt runs cycled an 8-prompt hot set at 80% frequency, yielding a 71% average cache hit rate.

Appendix B: Slot-Count Ablation

Increasing slots from 4 to 8 under cache-aware dispatching actually *reduces* performance: throughput falls to 0.699 req/s and the cache hit rate drops to 67%. The cache starts cold, so 8 simultaneous misses flood Ollama before any result reaches Redis; with 4 slots, fewer cold misses compete and the cache warms faster. Optimal slot count depends on workload repetition rate and warm-up dynamics.

References

[1] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SOSP*, 2023, pp. 611–626.

[2] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," in *Advances in NeurIPS 37*, 2024.

[3] Ollama, "Get up and running with large language models locally," `https://ollama.com`, accessed April 2026.